Documentation of Understanding

Anton — AI Agent Orchestration Platform

Aman Pradhan · Yuno AI Engineer Assessment

How I Read the Problem

The spec says "build a platform where users can create AI agents, configure how they behave, and connect them into collaborative workflows." That sentence contains three distinct problems that are easy to conflate:

1. **A configuration platform** — UI + persistence for defining agents and wiring them together
2. **A runtime** — something that actually executes agent logic, not a UI mockup
3. **A feedback mechanism** — a way for humans to trigger work and see results in real time

Most candidates will build (1) and fake (2). The spec explicitly rules that out: *"The chosen runtime must actually execute agent logic — not a UI mockup."*

I decided the right test of "real runtime" is: can I give it a hard research question, walk away, and come back to a professional output — without touching the system? If yes, the runtime is real. If I need to baby-sit it, it isn't.

That framing drove every architectural decision that followed.

The Core Design Decision: Stateful Graph vs. Simple Chain

The most important choice was the shape of the agent pipeline.

A simple chain — Orchestrator → Researcher → Analyst → Reporter — is easy to build and easy to demo. But it has a fatal flaw: there's no quality gate. The pipeline produces output regardless of whether that output is good.

Real intelligence workflows have revision cycles. An analyst produces a draft. A reviewer finds gaps. The analyst revises. This is how consulting firms, research teams, and journalists actually work.

I modeled this explicitly with a **Critic → Researcher feedback loop**. The Critic scores the analysis on a 1–10 rubric. If the score is below 7, the work goes back to the Researcher with specific gaps identified. This can repeat up to 3 times before the pipeline forces completion.

This is why I chose **LangGraph** over CrewAI or AutoGen:

- **LangGraph** supports cyclic graphs with explicit conditional edges. The Critic's routing decision (`approved → Reporter` , `rejected → Researcher`) is a first-class primitive.
- **CrewAI** abstracts routing behind high-level task assignment. You can't cleanly express "go back to step 2 if this condition is true."
- **AutoGen** is designed for conversational back-and-forth between agents. It fits chatbot-style systems, not structured pipelines with defined state schemas.

The feedback loop isn't a gimmick. In testing, it measurably improved output quality — runs that went through a revision cycle scored 0.8–1.2 points higher on the automated eval.

Why a Shared State TypedDict, Not Message Passing

LangGraph lets you choose between agents communicating via messages or via a shared state object. I chose shared state (`AgentState` TypedDict) for this pipeline.

The reason: in a research pipeline, every agent needs to see the full context — not just what the previous agent said. The Analyst needs the raw research data, not a summary. The Critic needs the analysis, not a description of the analysis. The Reporter needs everything.

Message passing would require each agent to either re-summarize context (lossy) or forward the entire conversation (wasteful). Shared state is the right model when agents are collaborating on a single artifact, not having a conversation.

```
class AgentState(TypedDict):
    run_id: str
    input: str
    search_queries: list[str]
    research_data: list[dict]
    analysis: str
    critique: str
    critique_approved: bool
    iteration: int
    report: str
    final_output: str
    messages: list
    token_counts: dict[str, int]
```

Every field is intentional. `iteration` prevents infinite loops. `critique_approved` is the conditional edge signal. `token_counts` per agent enables cost attribution, not just total cost.

Real-Time Events: Why Redis Pub/Sub

The Monitor tab needs to show agent activity as it happens. The naive approach is polling the database. The problem: agents write to the DB at completion, not during execution. You'd see nothing until the pipeline finishes.

I separated the event stream from the persistence layer:

- **Redis Pub/Sub** — ephemeral, low-latency, broadcast. Every agent publishes events as they happen (start, tool calls, completion). The frontend receives these within milliseconds.
- **Redis List** — a persistent buffer of the same events. New connections replay missed events from the buffer before subscribing to live events.
- **PostgreSQL** — final state only. Run record, messages, eval scores.

This pattern (pub/sub for live + list for replay) eliminates the race condition where a client connects after some events have already been published. Subscribe first, replay buffer, then receive live events — in that order.

One non-obvious bug I had to fix: individual agents emit `event_type="complete"` when their own step finishes. The terminal signal for the entire pipeline is `agent="system", event_type="complete"`. Conflating the two caused the frontend to stop polling after the Orchestrator finished, leaving all subsequent events invisible until page refresh. The fix was a one-line predicate change, but finding it required understanding the full event taxonomy.

The Eval System: Why It Exists

Most demos show a pipeline producing output. Few show a pipeline that knows whether its output is good.

I added an **LLM-as-judge evaluator** that runs automatically after every pipeline completion. It scores the report on four dimensions: Specificity, Completeness, Accuracy Risk, and Usefulness. Scores are stored in PostgreSQL and visible in the dashboard per run.

This serves two purposes:

1. **Product quality signal** — if scores are consistently low on one dimension, that points to a specific agent prompt needing improvement
2. **Regression gate** — before changing any agent prompt, run the batch eval endpoint (`POST /api/evals/run`) against 5 fixed test cases. If pass rate drops, the change

made things worse

The evaluator runs as a true background task (`asyncio.create_task`) so it doesn't delay the Telegram reply to the user. This was a deliberate latency optimization — the human gets their summary in ~45 seconds, and the eval score appears in the dashboard a few seconds later.

What I Chose Not to Build (and Why)

Per-agent model selection in the UI — the schema supports it, the agents are wired to config constants. I didn't expose model-per-agent in the UI because the more valuable tradeoff story (Flash vs. Pro for different reasoning depths) is better told in code than in a dropdown.

Schedule-triggered runs — the DB schema has a `schedule` field. I didn't build a cron-style scheduler because it would have been infrastructure theater — a scheduler that fires every N hours and hits the same pipeline isn't demonstrably more interesting than a Telegram trigger. I'd build it with APScheduler in a production version.

Content guardrails at runtime — the agent schema has `guardrails` and `max_tokens` fields. Runtime enforcement (prompt injection detection, output length hard stops) wasn't implemented. These are a next layer I'd build with LangChain's output parsers and a custom validation node in the graph.

Latency Optimizations Made

The initial pipeline took ~75 seconds end-to-end. Current average is ~45 seconds. Changes made:

Change	Saving
Orchestrator: Gemini 2.5 Pro → Flash	~12s (query generation doesn't need deep reasoning)
Researcher: parallel Tavily searches via `asyncio.gather`	~8s
Evaluator: `await` → `asyncio.create_task`	~6s off user-perceived latency
DB commit race fix: `flush()` → `commit()` before background task	Eliminated ~2s occasional startup stall

---

Architecture in One Paragraph

A FastAPI backend receives a Telegram webhook, creates a Run record in PostgreSQL (202 immediately), and fires a LangGraph pipeline as a background task. The pipeline runs 5 agents in a stateful graph with a conditional feedback loop. Every agent publishes events to Redis as it works. The Next.js frontend polls the Redis event buffer every 2 seconds and renders events as they arrive. When the pipeline completes, the Reporter sends a summary back to Telegram and the Evaluator scores the output asynchronously. Everything runs in Docker Compose with a single `make run`.

What I'd Build Next

1. **Multi-workflow routing** — today one workflow handles all Telegram messages. A dispatcher agent could route to different pipelines based on message intent.
2. **Agent memory** — store prior run summaries per user in a vector DB. The Orchestrator could use retrieval to avoid re-researching topics already covered.
3. **Runtime guardrails** — a validation node after each agent that checks output against schema and length constraints before passing state forward.
4. **Webhook reliability** — add a dead letter queue for failed Telegram sends and a retry mechanism for pipeline timeouts.
5. **Multi-channel** — the `BaseChannel` abstraction is already in place. Adding WhatsApp (via Twilio) or Slack is a single new class implementing `send_message` and `parse_incoming`.

Repository: github.com/Amanpradhan/anton Demo: loom.com/share/5d394b36906b447780f2bd1ab800de4a